Self Organizing Maps: Fundamentals

Introduction to Neural Networks : Lecture 16

© John A. Bullinaria, 2004

1. What is a Self Organizing Map?
2. Topographic Maps
3. Setting up a Self Organizing Map
4. Kohonen Networks
5. Components of Self Organization
6. Overview of the SOM Algorithm

Self Organizing Maps: Algorithms and Applications

Introduction to Neural Networks : Lecture 17

© John A. Bullinaria, 2004

1. The SOM Architecture and Algorithm
2. Properties of the Feature Map
   - Approximation of the Input Space
   - Topological Ordering
   - Density Matching
   - Feature Selection
3. Application: The Phonetic Typewriter

What is a Self Organizing Map?

So far we have looked at networks with ***supervised training*** techniques, in which there is a target output for each input pattern, and the network learns to produce the required outputs.

We now turn to ***unsupervised training***, in which the networks learn to form their own classifications of the training data without external help. To do this we have to assume that class membership is broadly defined by the input patterns sharing ***common features***, and that the network will be able to identify those features across the range of input patterns.

One particularly interesting class of unsupervised system is based on ***competitive learning***, in which the output neurons compete amongst themselves to be activated, with the result that only one is activated at any one time. This activated neuron is called a ***winner-takes-all neuron*** or simply the ***winning neuron***. Such competition can be induced/implemented by having ***lateral inhibition connections*** (negative feedback paths) between the neurons. The result is that the neurons are forced to organise themselves. For obvious reasons, such a network is called a ***Self Organizing Map (SOM)***.

Topographic Maps

Neurobiological studies indicate that different sensory inputs (motor, visual, auditory, etc.) are mapped onto corresponding areas of the cerebral cortex in an ***orderly fashion***.

This form of map, known as a ***topographic map***, has two important properties:

1. At each stage of representation, or processing, each piece of incoming information is kept in its proper context/neighbourhood.
2. Neurons dealing with closely related pieces of information are kept close together so that they can interact via short synaptic connections.

Our interest is in building artificial topographic maps that learn through self-organization in a neurobiologically inspired manner.

We shall follow the ***principle of topographic map formation***: "The spatial location of an output neuron in a topographic map corresponds to a particular domain or feature drawn from the input space".

Setting up a Self Organizing Map

The principal goal of an SOM is to ***transform*** an incoming signal pattern of arbitrary dimension into a one or two dimensional discrete map, and to perform this transformation adaptively in a topologically ordered fashion.

We therefore set up our SOM by placing neurons at the nodes of a one or two dimensional lattice. Higher dimensional maps are also possible, but not so common.

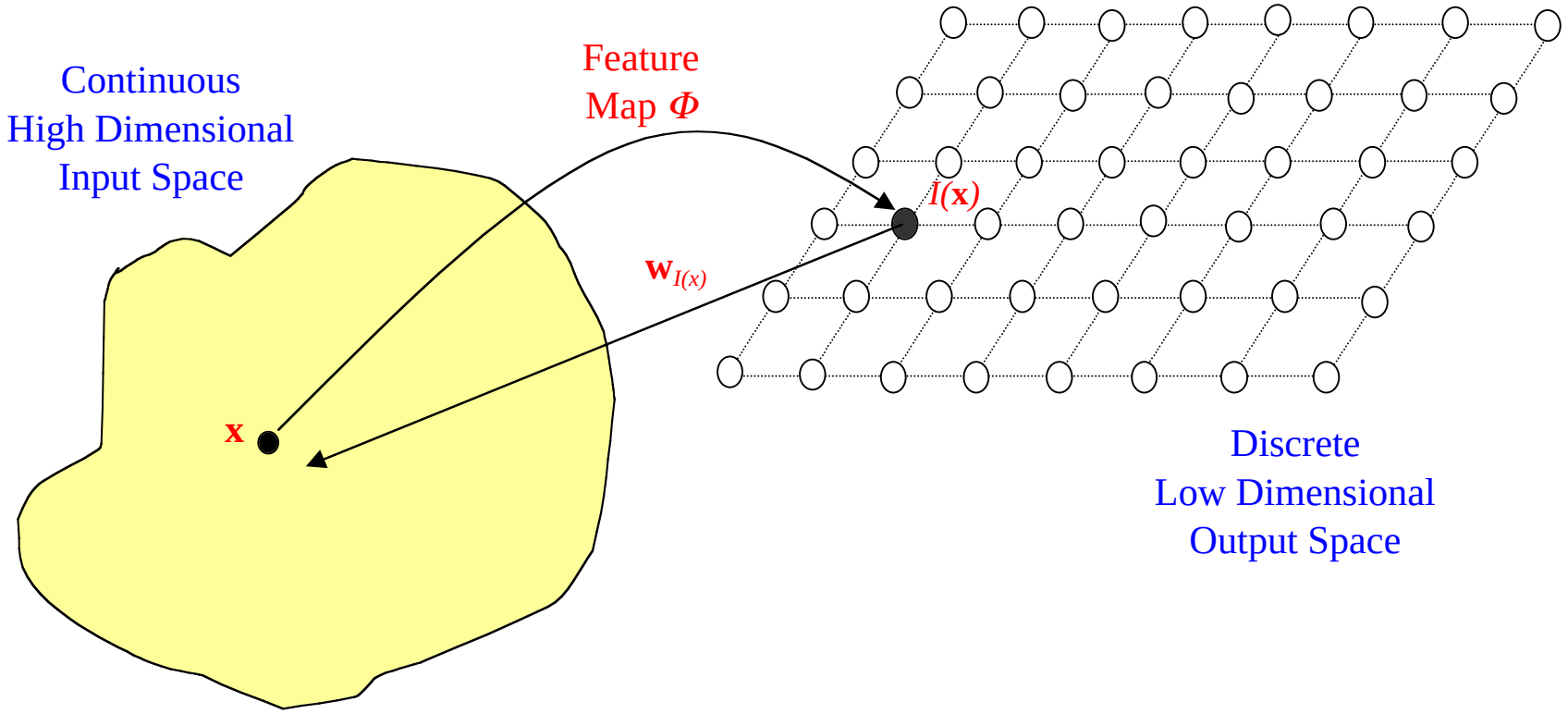
The neurons become ***selectively tuned*** to various input patterns (stimuli) or classes of input patterns during the course of the competitive learning.

The locations of the neurons so tuned (i.e. the winning neurons) become ordered and a meaningful ***coordinate system*** for the ***input features*** is created on the lattice. The SOM thus forms the required topographic map of the input patterns.

We can view this as a non-linear generalization of principal component analysis (PCA).

Organization of the Mapping

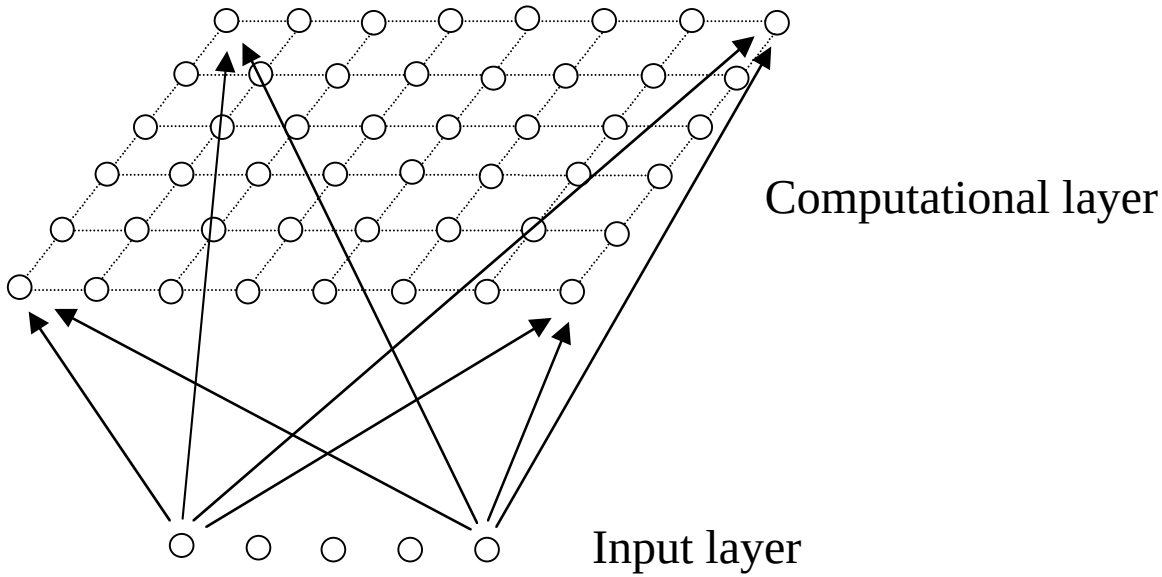
We have points **x** in the input space mapping to points $I(\mathbf{x})$ in the output space:



Each point I in the output space will map to a corresponding point $\mathbf{w}(I)$ in the input space.

Kohonen Networks

We shall concentrate on the particular kind of SOM known as a ***Kohonen Network***. This SOM has a feed-forward structure with a single computational layer arranged in rows and columns. Each neuron is fully connected to all the source nodes in the input layer:



Clearly, a one dimensional map will just have a single row (or a single column) in the computational layer.

Components of Self Organization

The self-organization process involves four major components:

Initialization: All the connection weights are initialized with small random values.

Competition: For each input pattern, the neurons compute their respective values of a *discriminant function* which provides the basis for competition. The particular neuron with the smallest value of the discriminant function is declared the winner.

Cooperation: The winning neuron determines the spatial location of a topological neighbourhood of excited neurons, thereby providing the basis for cooperation among neighbouring neurons.

Adaptation: The excited neurons decrease their individual values of the discriminant function in relation to the input pattern through suitable adjustment of the associated connection weights, such that the response of the winning neuron to the subsequent application of a similar input pattern is enhanced.

The Competitive Process

If the input space is D dimensional (i.e. there are D input units) we can write the input patterns as $\mathbf{x} = \{x_i : i = 1, \ldots, D\}$ and the connection weights between the input units i and the neurons j in the computation layer can be written $\mathbf{w}_j = \{w_{ji} : j = 1, \ldots, N;\ i = 1, \ldots, D\}$ where N is the total number of neurons.

We can then define our ***discriminant function*** to be the squared Euclidean distance between the input vector $\mathbf{x}$ and the weight vector $\mathbf{w}_j$ for each neuron j

$$d_j(\mathbf{x}) = \sum_{i=1}^{D} (x_i - w_{ji})^2$$

In other words, the neuron whose weight vector comes closest to the input vector (i.e. is most similar to it) is declared the winner.

In this way the continuous input space can be mapped to the discrete output space of neurons by a simple process of competition between the neurons.

The Cooperative Process

In neurobiological studies we find that there is ***lateral interaction*** within a set of excited neurons. When one neuron fires, its closest neighbours tend to get excited more than those further away. There is a ***topological neighbourhood*** that decays with distance.

We want to define a similar topological neighbourhood for the neurons in our SOM. If S_{ij} is the lateral distance between neurons i and j on the grid of neurons, we take

$$T_{j,I(\mathbf{x})} = \exp(-S_{j,I(\mathbf{x})}^2 / 2\sigma^2)$$

as our topological neighbourhood, where $I(\mathbf{x})$ is the index of the winning neuron. This has several important properties: it is maximal at the winning neuron, it is symmetrical about that neuron, it decreases monotonically to zero as the distance goes to infinity, and it is translation invariant (i.e. independent of the location of the winning neuron)

A special feature of the SOM is that the size σ of the neighbourhood needs to decrease with time. A popular time dependence is an exponential decay: $\sigma(t) = \sigma_0 \exp(-t / \tau_\sigma)$.

The Adaptive Process

Clearly our SOM must involve some kind of adaptive, or learning, process by which the outputs become self-organised and the ***feature map*** between inputs and outputs is formed.

The point of the topographic neighbourhood is that not only the winning neuron gets its weights updated, but its neighbours will have their weights updated as well, although by not as much as the winner itself. In practice, the appropriate weight update equation is

$$\Delta w_{ji} = \eta(t) \,.\, T_{j,I(\mathbf{x})}(t) \,.\, (x_i - w_{ji})$$

in which we have a time (epoch) t dependent learning rate $\eta(t) = \eta_0 \exp(-t/\tau_\eta)$, and the updates are applied for all the training patterns $\mathbf{x}$ over many epochs.

The effect of each learning weight update is to move the weight vectors $\mathbf{w}_i$ of the winning neuron and its neighbours towards the input vector $\mathbf{x}$. Repeated presentations of the training data thus leads to topological ordering.

Ordering and Convergence

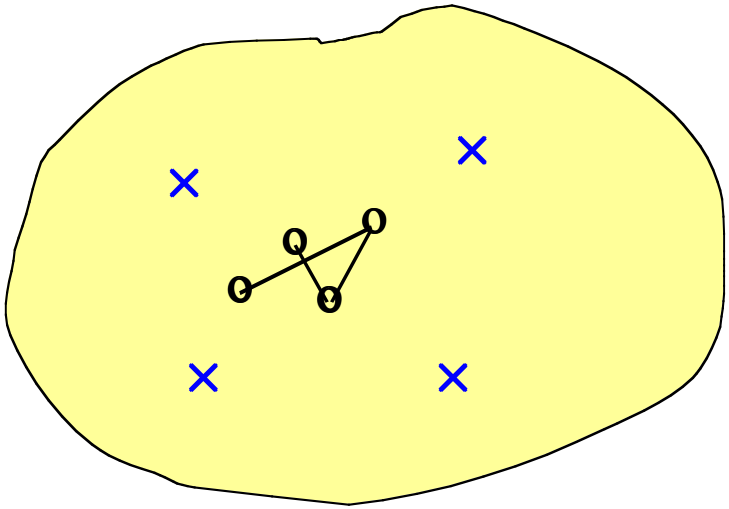
Provided the parameters (σ_0, τ_σ, η_0, τ_η) are selected properly, we can start from an initial state of complete disorder, and the SOM algorithm will gradually lead to an organized representation of activation patterns drawn from the input space. (However, it is possible to end up in a ***metastable state*** in which the feature map has a topological defect.)

There are two identifiable phases of this adaptive process:

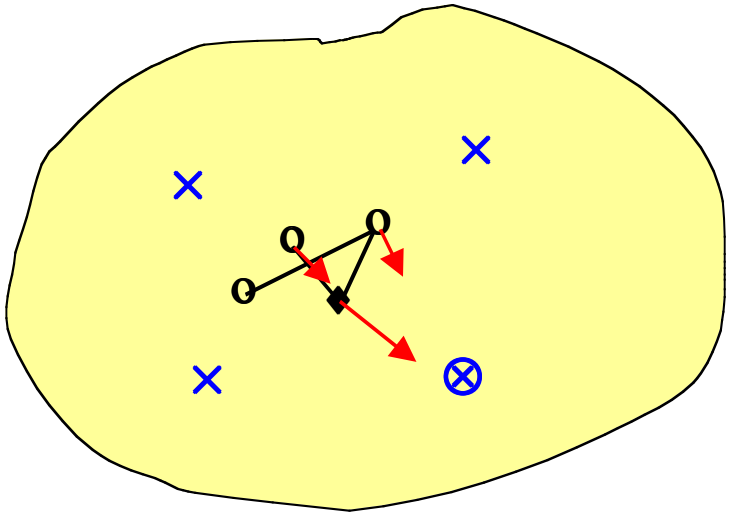
1. **Ordering or self-organizing phase** – during which the topological ordering of the weight vectors takes place. Typically this will take as many as 1000 iterations of the SOM algorithm, and careful consideration needs to be given to the choice of neighbourhood and learning rate parameters.

2. **Convergence phase** – during which the feature map is fine tuned and comes to provide an accurate statistical quantification of the input space. Typically the number of iterations in this phase will be at least 500 times the number of neurons in the network, and again the parameters must be chosen carefully.

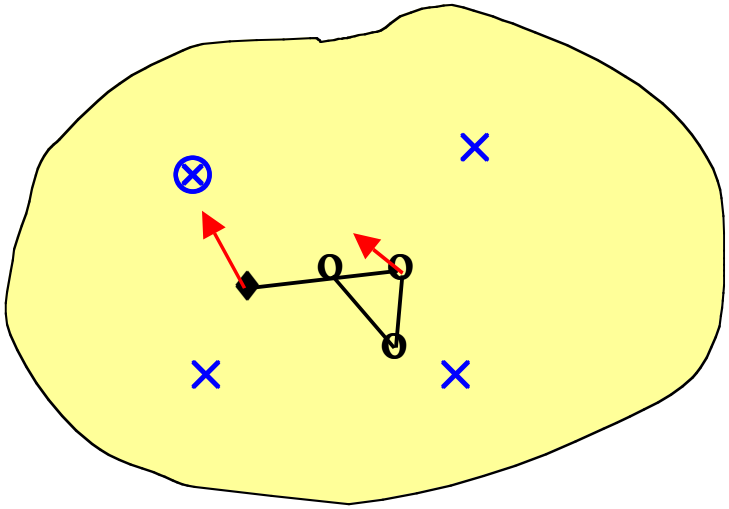
Visualizing the Self Organization Process



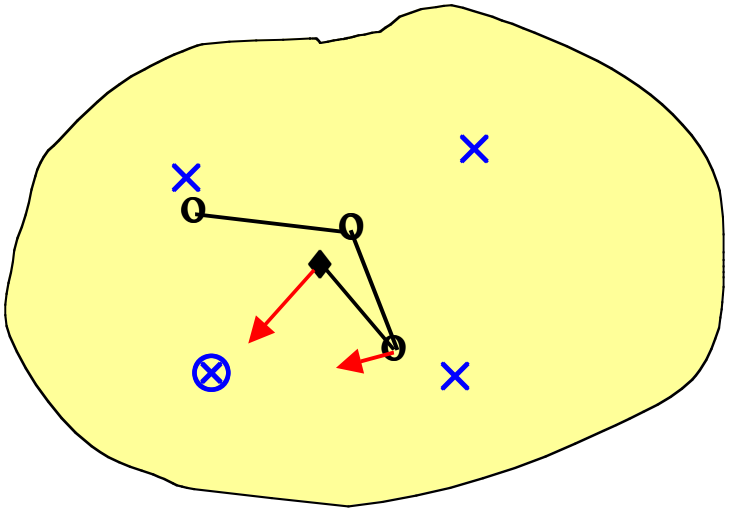
Suppose we have four data points (crosses) in our continuous 2D input space, and want to map this onto four points in a discrete 1D output space. The output nodes map to points in the input space (circles). Random initial weights start the circles at random positions in the centre of the input space.



We randomly pick one of the data points for training (cross in circle). The closest output point represents the winning neuron (solid diamond). That winning neuron is moved towards the data point by a certain amount, and the two neighbouring neurons move by smaller amounts (arrows).



Next we randomly pick another data point for training (cross in circle). The closest output point gives the new winning neuron (solid diamond). The winning neuron moves towards the data point by a certain amount, and the one neighbouring neuron moves by a smaller amount (arrows).



We carry on randomly picking data points for training (cross in circle). Each winning neuron moves towards the data point by a certain amount, and its neighbouring neuron(s) move by smaller amounts (arrows). Eventually the whole output grid unravels itself to represent the input space.

Overview of the SOM Algorithm

We have a spatially ***continuous input space***, in which our input vectors live. The aim is to map from this to a low dimensional spatially ***discrete output space***, the topology of which is formed by arranging a set of neurons in a grid. Our SOM provides such a non-linear transformation called a ***feature map***.

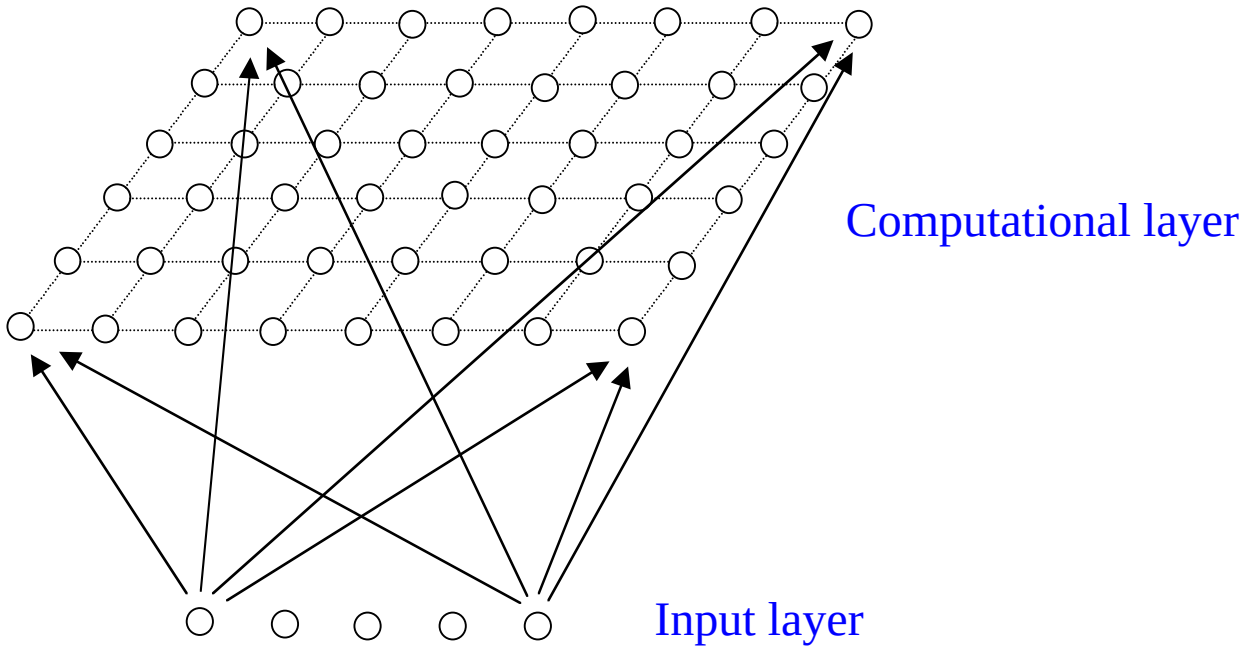
The stages of the SOM algorithm can be summarised as follows:

1. ***Initialization*** – Choose random values for the initial weight vectors $\mathbf{w}_j$.
2. ***Sampling*** – Draw a sample training input vector $\mathbf{x}$ from the input space.
3. ***Matching*** – Find the winning neuron $I(\mathbf{x})$ with weight vector closest to input vector.
4. ***Updating*** – Apply the weight update equation $\Delta w_{ji} = \eta(t)\, T_{j,I(\mathbf{x})}(t)\, (x_i - w_{ji})$.
5. ***Continuation*** – keep returning to step 2 until the feature map stops changing.

Next lecture we shall explore the properties of the resulting feature map and look at some simple examples of its application.

The Architecture a Self Organizing Map

We shall concentrate on the SOM system known as a ***Kohonen Network***. This has a feed-forward structure with a single computational layer of neurons arranged in rows and columns. Each neuron is fully connected to all the source units in the input layer:



A one dimensional map will just have a single row or column in the computational layer.

The SOM Algorithm

The aim is to learn a ***feature map*** from the spatially ***continuous input space***, in which our input vectors live, to the low dimensional spatially ***discrete output space*****,** which is formed by arranging the computational neurons into a grid.

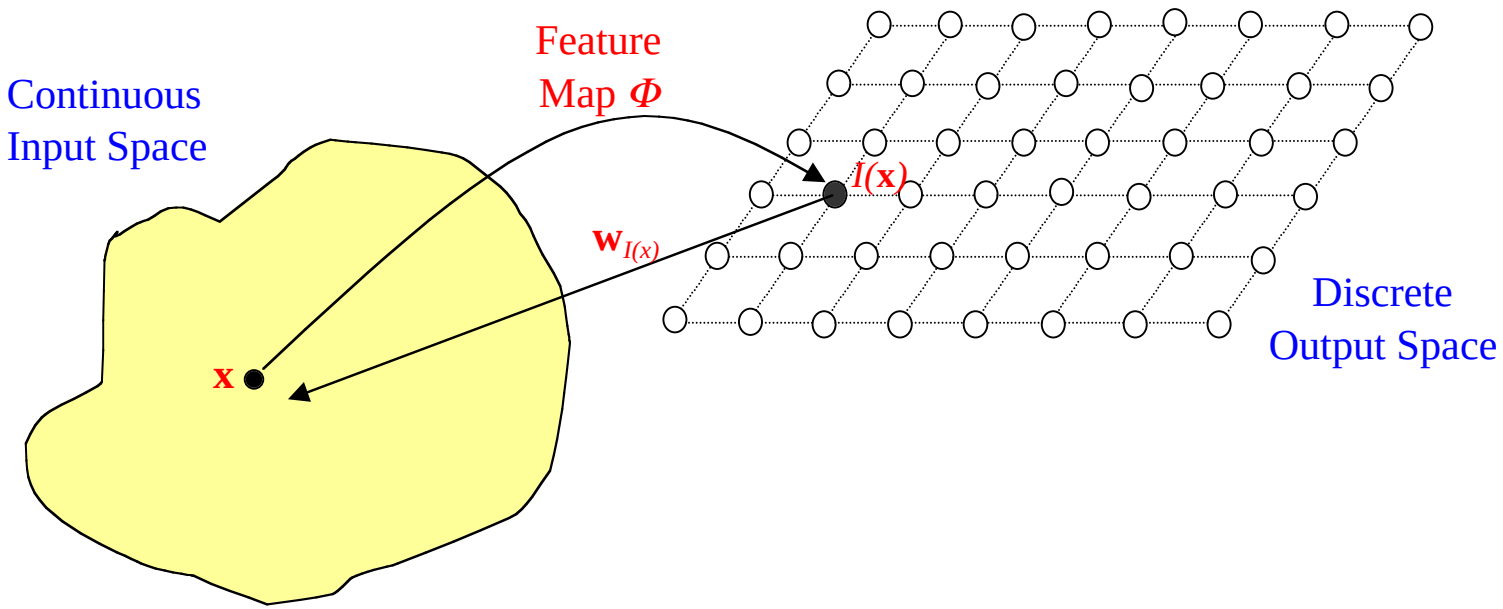
The stages of the SOM algorithm that achieves this can be summarised as follows:

1. ***Initialization*** – Choose random values for the initial weight vectors $\mathbf{w}_j$.
2. ***Sampling*** – Draw a sample training input vector $\mathbf{x}$ from the input space.
3. ***Matching*** – Find the winning neuron $I(\mathbf{x})$ that has weight vector closest to the input vector, i.e. the minimum value of $d_j(\mathbf{x}) = \sum_{i=1}^{D}(x_i - w_{ji})^2$.
4. ***Updating*** – Apply the weight update equation $\Delta w_{ji} = \eta(t)\, T_{j,I(\mathbf{x})}(t)\, (x_i - w_{ji})$ where $T_{j,I(\mathbf{x})}(t)$ is a Gaussian neighbourhood and $\eta(t)$ is the learning rate.
5. ***Continuation*** – keep returning to step 2 until the feature map stops changing.

We shall now explore the properties of the feature map and look at some examples.

Properties of the Feature Map

Once the SOM algorithm has converged, the feature map displays important statistical characteristics of the input space. Given an input vector **x**, the feature map Φ provides a winning neuron $I(\mathbf{x})$ in the output space, and the weight vector $\mathbf{w}_{I(\mathbf{x})}$ provides the coordinates of the image of that neuron in the input space.



The feature map has four important properties that we shall look at in turn:

Property 1 : Approximation of the Input Space

The feature map Φ represented by the set of weight vectors $\{\mathbf{w}_i\}$ in the output space, provides a good approximation to the input space.

We can state the aim of the SOM as storing a large set of input vectors $\{\mathbf{x}\}$ by finding a smaller set of prototypes $\{\mathbf{w}_i\}$ so as to provide a good approximation to the original input space. The theoretical basis of this idea is rooted in ***vector quantization theory***, the motivation of which is dimensionality reduction or data compression.

In effect, the goodness of the approximation is given by the total squared distance

$$D = \sum_{\mathbf{x}} \left\| \mathbf{x} - \mathbf{w}_{I(\mathbf{x})} \right\|^2$$

which we wish to minimize. If we work through gradient descent style mathematics we do end up with the SOM weight update algorithm, which confirms that it is generating a good approximation to the input space.

Property 2 : Topological Ordering

The feature map Φ computed by the SOM algorithm is topologically ordered in the sense that the spatial location of a neuron in the output lattice/grid corresponds to a particular domain or feature of the input patterns.

The topological ordering property is a direct consequence of the weight update equation that forces the weight vector $\mathbf{w}_{I(\mathbf{x})}$ of the winning neuron $I(\mathbf{x})$ to move toward the input vector $\mathbf{x}$. The crucial factor is that the weight updates also move the weight vectors $\mathbf{w}_j$ of the closest neighbouring neurons j along with the winning neuron $I(\mathbf{x})$. Together these weight changes cause the whole output space to become appropriately ordered.

We can visualise the feature map Φ as an ***elastic or virtual net*** with a grid like topology. Each output node can be represented in the input space at coordinates given by their weights. Then if the neighbouring nodes in output space have their corresponding points in input space connected together, the resulting image of the output grid reveals directly the topological ordering at each stage of the network training.

Property 3 : Density Matching

The feature map Φ reflects variations in the statistics of the input distribution: regions in the input space from which the sample training vectors x are drawn with high probability of occurrence are mapped onto larger domains of the output space, and therefore with better resolution than regions of input space from which training vectors are drawn with low probability.

We need to relate the input vector probability distribution $p(\mathbf{x})$ to the ***magnification factor*** $m(\mathbf{x})$ of the feature map. Generally, for two dimensional feature maps the relation cannot be expressed as a simple function, but in one dimension we can show that

$$m(\mathbf{x}) \propto p^{2/3}(\mathbf{x})$$

So the SOM algorithm doesn't match the input density exactly, because of the power of 2/3 rather than 1. Computer simulations indicate similar approximate density matching in general, always with the low input density regions slightly over-represented.

Property 4 : Feature Selection

Given data from an input space with a non-linear distribution, the self organizing map is able to select a set of best features for approximating the underlying distribution.

This property is a natural culmination of properties 1 through 3.

Remember how Principal Component Analysis (PCA) is able to compute the input dimensions which carry the most variance in the training data. It does this by computing the eigenvector associated with the largest eigenvalue of the correlation matrix.

PCA is fine if the data really does form a line or plane in input space, but if the data forms a curved line or surface (e.g. a semi-circle), linear PCA is no good, but a SOM will overcome the approximation problem by virtue of its topological ordering property.

The SOM provides a discrete approximation of finding so-called ***principal curves*** or ***principal surfaces***, and may therefore be viewed as a non-linear generalization of PCA.

Application: The Phonetic Typewriter

One of the earliest and well known applications of the SOM is the phonetic typewriter of Kohonen. It is set in the field of speech recognition, and the problem is to classify phonemes in real time so that they could be used to drive a typewriter from dictation.

The real speech signals obviously needed pre-processing before being applied to the SOM. A combination of filtering and Fourier transforming of data sampled every 9.83 ms from spoken words provided a set of 16 dimensional spectral vectors. These formed the input space of the SOM, and the output space was an 8 by 12 grid of nodes.

Though the network was effectively trained on time-sliced speech waveforms, the output nodes became sensitised to phonemes and the relations between, because the network inputs were real speech signals which are naturally clustered around ideal phonemes. As a spoken word is processed, a path through output space maps out a phonetic transcription of the word. Some post-processing was required because phonemes are typically 40-400 ms long and span many time slices, but the system was surprisingly good at producing sensible strings of phonemes from real speech.

Overview and Reading

1. We began by defining what we mean by a Self Organizing Map (SOM) and by a topographic map.
2. We then looked at how to set up a SOM and at the components of self organisation: competition, cooperation, and adaptation. We saw that the self organization has two identifiable stages: ordering and convergence.
3. We ended with an overview of the SOM algorithm and its five stages: initialization, sampling, matching, updating, and continuation.

Reading

1. Haykin: Sections 9.1, 9.2, 9.3, 9.4.
2. Beale & Jackson: Sections 5.1, 5.2, 5.3, 5.4, 5.5
3. Gurney: Sections 8.1, 8.2, 8.3
4. Hertz, Krogh & Palmer: Sections 9.4, 9.5
5. Callan: Sections 3.1, 3.2, 3.3, 3.4, 3.5

Overview and Reading

1. We began by reviewing the SOM architecture and algorithm.
2. We then looked at the important properties of the feature map: its ability to approximate the input space, the topological ordering that emerges, the matching of the input space probability densities, and the ability to select the best set of features for approximating the underlying input distribution.
3. We ended with a simple application – the phonetic typewriter.

Reading

1. Haykin: Sections 9.3, 9.4, 9.5, 9.6
2. Beale & Jackson: Sections 5.2, 5.3, 5.4, 5.5, 5.7
3. Gurney: Sections 8.1, 8.2, 8.3
4. Hertz, Krogh & Palmer: Sections 9.4, 9.5
5. Callan: Sections 3.1, 3.2, 3.3, 3.4, 3.5